

SARGEtools Example Markdown

Stephanie J. Wilson

Last Updated: 09-29-2026

This code is an example or template for creating a pdf summary of SARGE data. With this code you can plot data using the `sarge_plot_defaults()` function which will plot any data set as a default parameter

Step 1: Set Up R to run this Code

Download the SARGEtools package if you do not already have it

```
#Use remotes to install the SARGEtools package from Github

#if you don't have the remotes package use:
  #install.packages("remotes")
library(remotes)

# Install from GitHub
remotes::install_github("Smithsonian/SARGEtools")
```

Read in packages that will be needed to run this code

```
knitr::opts_chunk$set(echo = TRUE)

#Load required packages to run this markdown

library(tidyverse)
library(lubridate)
library(plotly)
library(DT)
library(SARGEtools)
```

Set the time zone and date range of interest

```
# Set TIMEZONE that is relevant to your data
TIMEZONE = "EST"
```

```
# Define the date window you want shown on the plots
start_date = "2024-09-20"
end_date = "2024-09-27"
```

Step 2: Set your SARGE data configuration

Define the data file descriptions

File name in example: `file_descriptions.txt` or `file_descriptions.csv`

User builds this file!

Need to set up a file that has links to your data files output by SARGE (these can be online in a Dropbox, Google Drive, Sharepoint, GitHub, etc). If you are not using an online repository, you can upload your files to a folder and put the path in the `file_tab_descriptions.txt`, but if you have an online repository you can put the link to the data. Note: If you are using google sheets, make the sheet so anyone with the link can view the file, then use the `googlesheets` package and add this line to your code: `googlesheets4::gs4_deauth()`

The `file_descriptions` will include user defined columns:

- **data_type** = data type descriptor (e.g. Physiochemical, Hydrologic, GHG, Radon) need one for each data link, so typically one per instrument.
- **data_link** = location of the data (e.g. web link or file path).
- **skip** = how many lines will need to be skipped before reaching data in the file.
- **colname_override** = these are the column names that we are defining for the linked file.

It is important to verify that each of these columns and rows match the data coming from the instrument of interest.

Data the column name descriptions file. This describes your variables, renames instrument or campbell code defined columns.

File name in example: `colname_descriptions.csv`

User builds this file!

The `variable_descriptions` file will include user defined columns:

- **data_type** = data type descriptor (e.g. Physiochemical, Hydrologic, GHG, Radon) need one for each data link, should match `file_descriptions` `data_types`.
- **label** = label that will be used when plotting the data.
- **colname** = column name that will match one of the listed values in `colname_override` for the specified `data_type`.
- **color** = this is the R color that the variable will be plotted in.
- **default** = determines if this is a default plotting variable (T) or not (F); `sarge_plot_defaults` will only plot those listed as "T".

```
## Define your file_descriptions file and read it:

# If you are using a text file:
files_txt <- read_delim(
  #enter your path and file name:
  "../template_markdown/configure_data/file_descriptions.txt",

  show_col_types = FALSE, delim = "\t")

# If you are using a csv file:
files_csv <- read_csv(
  #enter your path and file name:
  "../template_markdown/configure_data/file_descriptions.csv",

  show_col_types = FALSE)

# Define your colname_descriptions file and read it:
variables <- read_csv(
  #enter your path and file name:
  "../template_markdown/configure_data/variable_descriptions.csv",

  show_col_types = FALSE) %>%

mutate(
  default = as.logical(default) ) %>%

mutate(colname = if_else(is.na(colname), label, colname),

       label = if_else(is.na(label), colname, label))
```

Step 3: Read in your SARGE Data

Use the `sarge_read()` function here to read in data listed in `file_descriptions`

```
# Read in the data with sarge_read() function
data <- sarge_read(
  files = files_csv,           # dataframe containing your file_descriptions from Step 2
  variables = variables,      # dataframe containing your variable_descriptions from Step 2
  timezone = TIMEZONE,        # TIMEZONE value defined in Step 1
  resolution = "minute"       # time resolution you would like (e.g. "minute", "hour")
)
```

```
## Reading SARGE data...
```

```
##   Timezone: EST
```

```
##   Resolution: minute
```

```
## Files found: 4

##

## Physiochemistry | SARGE_ExoTable.dat | 2024-08-15 17:24:00 → 2024-09-27 09:00:00

## Hydrology | SARGE_SontekTable.dat | 2024-08-19 09:39:00 → 2024-09-27 09:00:00

## GHG | SARGE_GHG_7810.dat | 2024-08-15 17:19:00 → 2024-09-27 09:00:00

## Radon | SARGE_Rad7Table.dat | 2024-08-15 17:30:00 → 2024-09-27 09:00:00

##

## SARGE data reading complete.
```

```
# Take a first look at the data
```

```
head(data)
```

```
## $Physiochemistry
## # A tibble: 3,679 x 11
##   TIMESTAMP          ODO_MgL FDOM_RFU Chlorophyll_ugL Conductivity ODO_sat
##   <dtm>              <dbl>   <dbl>          <dbl>          <dbl>   <dbl>
## 1 2024-08-15 17:24:00    8.31    17.2            0            3.54   109.
## 2 2024-08-15 17:30:00    8.25    17.8            0            3.51   108.
## 3 2024-08-15 17:36:00    8.22    17.9            0            3.49   108.
## 4 2024-08-15 17:42:00    8.18    18.0            0            3.46   107.
## 5 2024-08-15 17:48:00    8.15    18.4            0            3.44   107.
## 6 2024-08-15 17:54:00    8.1     17.4            0            3.43   106.
## 7 2024-08-15 18:00:00    8.03    18.1            0            3.43   105.
## 8 2024-08-15 18:06:00    7.91    17.3            0            3.44   103.
## 9 2024-08-15 18:12:00    7.77    17.2            0            3.45   102.
## 10 2024-08-15 18:18:00    7.62    16.6            0            3.50   99.6
## # i 3,669 more rows
## # i 5 more variables: Salinity_PPT <dbl>, TDS_mg_L <dbl>, pH <dbl>,
## #   Temp_C <dbl>, Depth_m <dbl>
##
## $Hydrology
## # A tibble: 5,679 x 5
##   TIMESTAMP          Water_depth Flowrate Index_velocity Mean_velocity
##   <dtm>              <dbl>   <dbl>          <dbl>          <dbl>
## 1 2024-08-19 09:39:00    0.855     0        -0.354            0
## 2 2024-08-19 09:42:00    0.855     0        -0.354            0
## 3 2024-08-19 09:45:00    0.867     0        -0.38             0
## 4 2024-08-19 09:48:00    0.867     0        -0.38             0
## 5 2024-08-19 09:51:00    0.884     0        -0.364            0
## 6 2024-08-19 09:54:00    0.884     0        -0.364            0
## 7 2024-08-19 09:57:00    0.899     0        -0.358            0
## 8 2024-08-19 10:00:00    0.899     0        -0.358            0
## 9 2024-08-19 10:03:00    0.913     0        -0.343            0
## 10 2024-08-19 10:06:00    0.913     0        -0.343            0
```

```
## # i 5,669 more rows
##
## $GHG
## # A tibble: 22,194 x 6
##   TIMESTAMP          H2O    CO2    CH4 cavity_p cavity_t
##   <dtm>          <dbl> <dbl> <dbl>   <dbl>   <dbl>
## 1 2024-08-15 17:19:00 19679.  404. 2513.    40.1    55.0
## 2 2024-08-15 17:20:00 12794. 2260. 2612.    40.0    55.0
## 3 2024-08-15 17:21:00  8603. 2526. 2685.    40.0    55.0
## 4 2024-08-15 17:22:00  9680. 1869. 2741.    40.0    55.0
## 5 2024-08-15 17:23:00 10103. 2159. 2753.    40.0    55.0
## 6 2024-08-15 17:24:00  9535. 2237. 2727.    40.0    55.0
## 7 2024-08-15 17:25:00  8794. 2022. 2745.    40.0    55.0
## 8 2024-08-15 17:26:00  8062. 1716. 2813.    40.0    55.0
## 9 2024-08-15 17:27:00  7383. 1418. 2885.    40.0    55.0
## 10 2024-08-15 17:28:00  6777. 1171. 3024.    40.0    55.0
## # i 22,184 more rows
##
## $Radon
## # A tibble: 1,462 x 4
##   TIMESTAMP          RH_sample_Rad7 Radon_concentration_Rad7
##   <dtm>          <dbl>          <dbl>
## 1 2024-08-15 17:30:00           32           132
## 2 2024-08-15 17:45:00           32           132
## 3 2024-08-15 18:00:00           11           12.4
## 4 2024-08-15 18:15:00            8           49.2
## 5 2024-08-15 18:30:00            7           74.0
## 6 2024-08-15 18:45:00            6           37.0
## 7 2024-08-15 19:00:00            6          210.
## 8 2024-08-15 19:15:00            5           49.3
## 9 2024-08-15 19:30:00            5           74.1
## 10 2024-08-15 19:45:00            5           61.8
## # i 1,452 more rows
## # i 1 more variable: Radon_concentration_uncertainty_Rad7 <dbl>
```

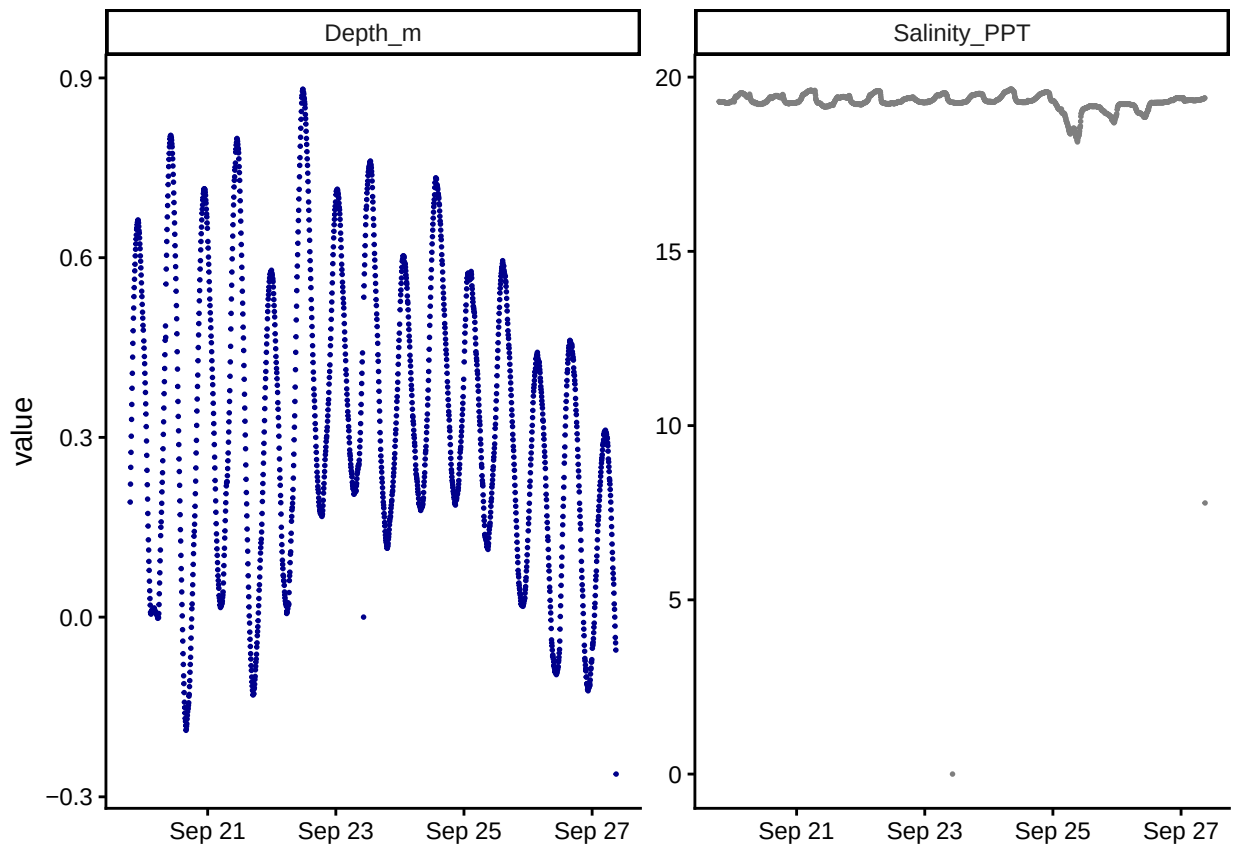
Step 4: Plot the SARGE Data

Physiochemical Data: Measurements made by the Xylem EXO2 Sonde in the example data

```
# Set the desired data_type from your data (must match a data_type in variables)
data_type = "Physiochemistry"

# Plot the greenhouse gas data from SARGE
exo_plot <- sarge_plot_defaults(
  data = data,                #file_descriptions dataframe
  data_type = data_type,      #data_type you want to plot
  start = start_date,         #the defined start date value
  end = end_date,             #the defined end date value
  variables = variables)      #variables dataframe with colnames, colors
```

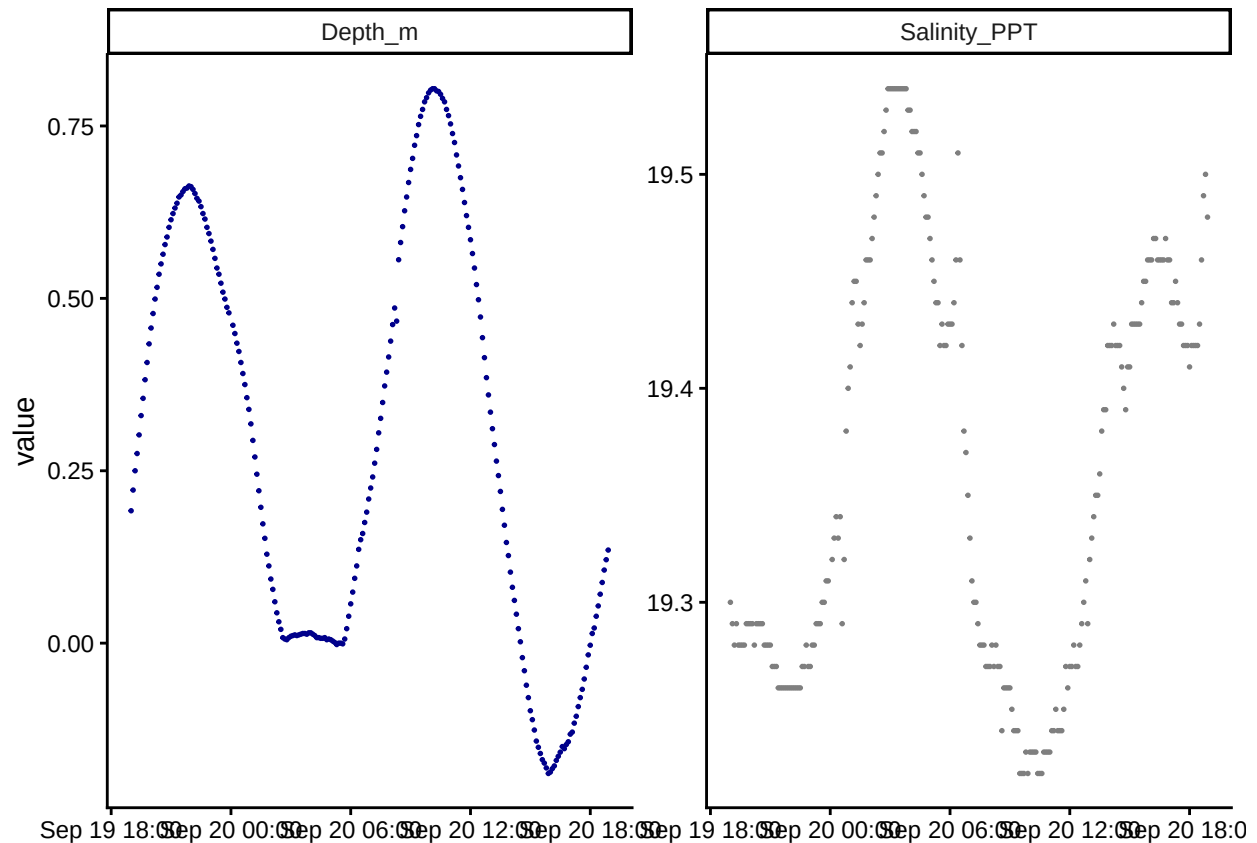
```
# Print the Plot
exo_plot
```



```
# Plot the data for a shorter window of time and defined it in the function
exo_plot1 <- sarge_plot_defaults(
  data = data,
  data_type = "Physiochemistry",
  start = "2024-09-20 12:00:00",
  end = "2024-09-20 14:00:00",
  variables = variables)

#file_descriptions dataframe
#data_type you want to plot
#newly defined start date
#newly defined end date
#variables dataframe with colnames, colors

# Print the Plot
exo_plot1
```



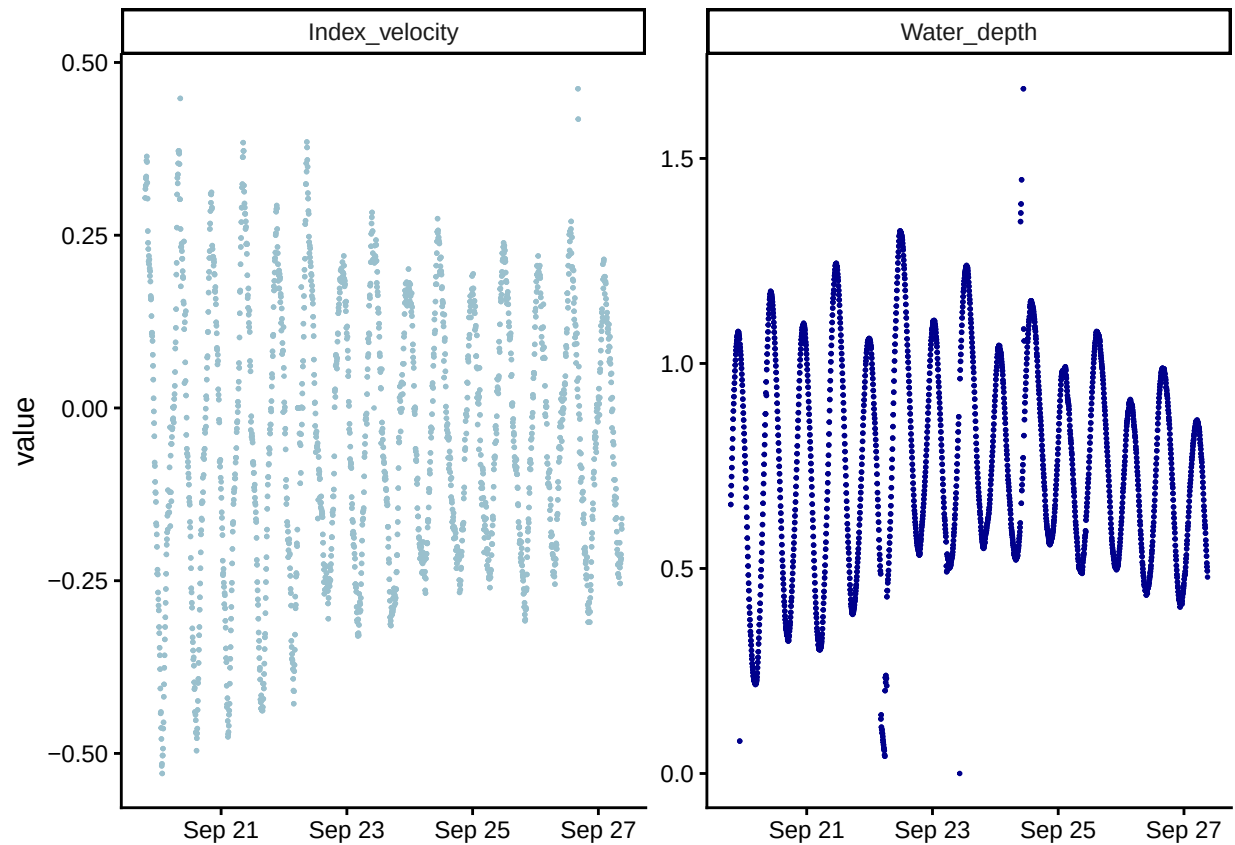
Hydrologic Data: Measurements made by the Sontek IQ ADCP in the example data

```
# Set the desired data_type from your data (must match a data_type in variables)
data_type = "Hydrology"

# Plot the greenhouse gas data from SARGE
adcp_plot <- sarge_plot_defaults(
  data = data,
  data_type = data_type,
  start = start_date,
  end = end_date,
  variables = variables)

#file_descriptions dataframe
#data_type you want to plot
#the defined start date value
#the defined end date value
#variables dataframe with colnames, colors

# Print the Plot
adcp_plot
```



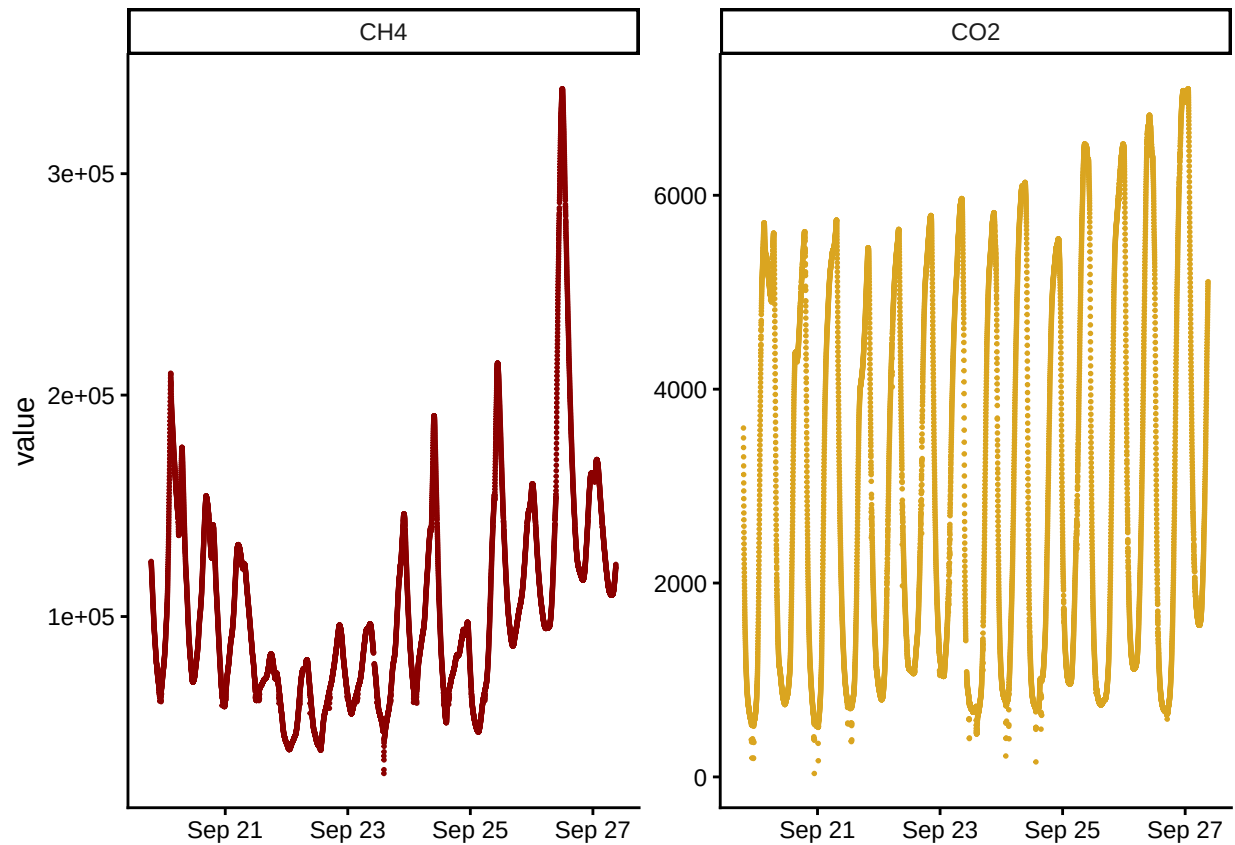
Greenhouse Gas Data: Measurements made by a LICOR 7810 in the example data

```
# Set the desired data_type from your data (must match a data_type in variables)
data_type = "GHG"

# Plot the greenhouse gas data from SARGE
ghg_plot <- sarge_plot_defaults(
  data = data,
  data_type = data_type,
  start = start_date,
  end = end_date,
  variables = variables)

#file_descriptions dataframe
#data_type you want to plot
#the defined start date value
#the defined end date value
#variables dataframe with colnames, colors

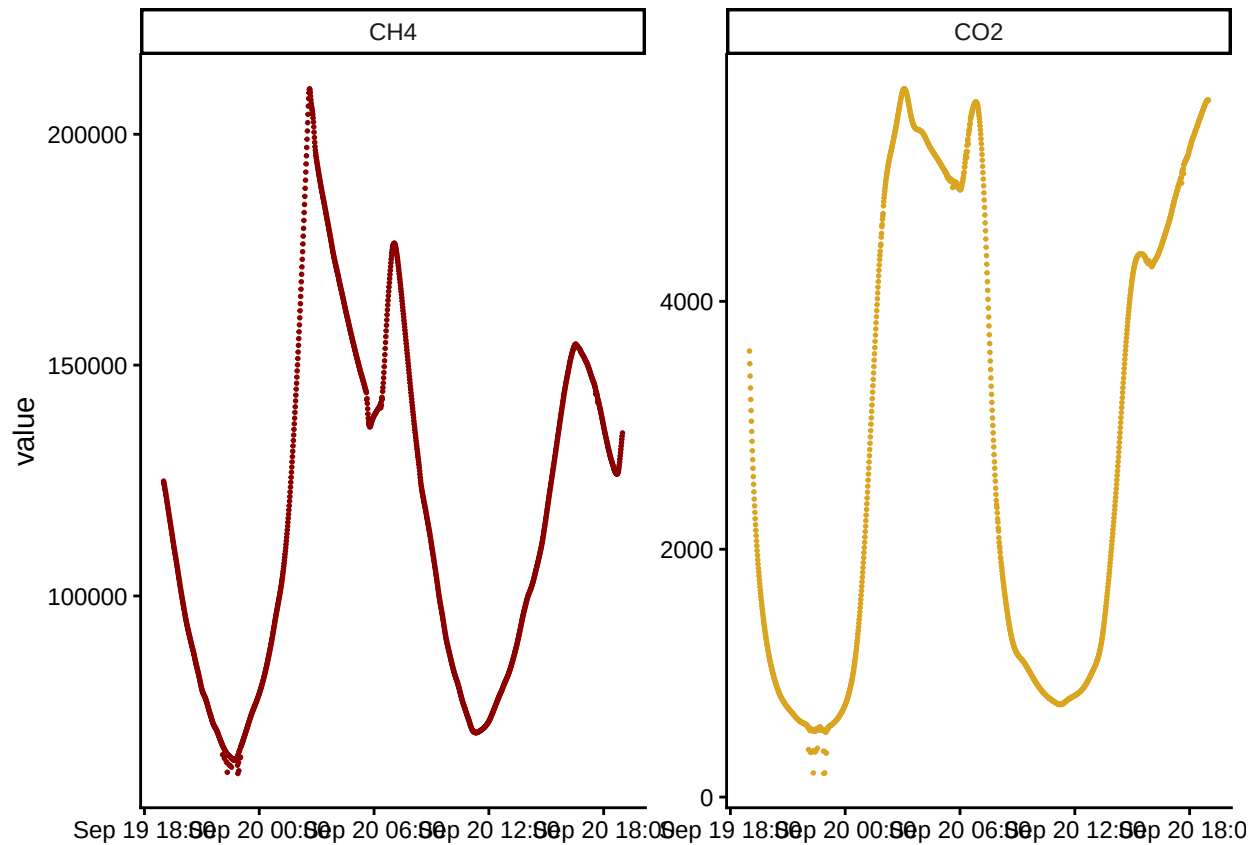
# Print the Plot
ghg_plot
```



```
# Plot the greenhouse gas data for a shorter window of time and defined it in the function
ghg_plot1 <- sarge_plot_defaults(
  data = data,
  data_type = "GHG",
  start = "2024-09-20 12:00:00",
  end = "2024-09-20 14:00:00",
  variables = variables)

#file_descriptions dataframe
#data_type you want to plot
#newly defined start date
#newly defined end date
#variables dataframe with colnames, colors

# Print the Plot
ghg_plot1
```



Radon Gas Data: Measurements made by a DurrIDGE RAD7 in the example data

```
# Set the desired data_type from your data (must match a data_type in variables)
data_type = "Radon"

# Plot the greenhouse gas data from SARGE
rad7_plot <- sarge_plot_defaults(
  data = data,
  data_type = data_type,
  start = start_date,
  end = end_date,
  variables = variables)

#file_descriptions dataframe
#data_type you want to plot
#the defined start date value
#the defined end date value
#variables dataframe with colnames, colors

# Print the Plot
rad7_plot
```

